SKN35 · LLM 연동 내외부 문서 기반 질의응답 시스템

LangChain 버전 마이그레이션 코드 어시스턴트

옛 버전 코드를 원하는 버전에서 돌아가는 코드로, 근거와 검증까지 함께

SKN35 3rd 4Team · 발표 [이름] · 2026-10-15

목차

15분 동안 이 순서로 말씀드리겠습니다

1	문제와 해결	LLM은 옛 코드를 주고 틀려도 모른다 → 근거 + 검사
2	만든 것	데이터 · 그래프 · 검증 11가지 · 화면 · 대화 기록
3	평가와 결과	질문 30개 비교 설계 · 실제 실행 사례 · 결과
4	배운 것	발견 · 한계 · 가치

ChatGPT가 준 LangChain 코드, 실행하면 깨집니다

```
# LLM이 흔히 주는 코드

from langchain.chat_models import ChatOpenAI
from langchain.chains import LLMChain

---

# LangChain 1.4.5에서 실행 결과

ImportError: cannot import name 'ChatOpenAI'
ModuleNotFoundError: No module named 'langchain.chains'
```

첫 줄부터 에러

LangChain 1.0에서 패키지가 나뉘고, 옛 체인과 에이전트가 빠졌습니다.

더 큰 문제

LLM은 자기 답이 틀렸다는 것을 **모릅니다.**

인터넷에는 옛 버전 코드가 더 많습니다

LLM은 데이터가 많은 쪽을 따라갑니다. 라이브러리는 빨리 바뀌어도, 학습 데이터 속 코드는 대부분 과거 버전입니다.

42.55%

문서가 부족할 때, LLM 코드 중 목표 버전에서 실행되는 비율

When LLMs Lag Behind, 2026

66.36%

구조화된 문서를 줘도 실행되는 비율

When LLMs Lag Behind, 2026

48~51%

최신 상용 모델의 버전 지정 코드 생성 성공률

GitChameleon 2.0, 2025

→ 문서를 주는 것만으로는 부족하고, 고친 코드를 확인하는 단계가 필요합니다.

근거로 고치고, 검사로 한 번 더 확인합니다

구분	일반 LLM	우리 챗봇
근거	학습 데이터 (옛 버전이 다수)	import · 메서드 대응표 + 최신 공식 문서 검색 (RAG)
확인	없음	verifier가 LLM 없이 11가지 검사 (문법, import 존재, 남은 옛 import …)
틀렸을 때	모름	실패 이유로 근거를 다시 찾고, 틀린 근거는 빼고 다시 고침 (최대 3회)
답변	코드만	고친 코드 + 바뀐 줄마다 근거 출처 · 설명 + 시도 횟수 + 설치할 패키지
버전	대부분 최신으로만	질문에 적은 버전으로 (적지 않으면 최신 1.4.5), 되돌리기 포함

고칠 때 볼 근거를 미리 만들어 두었습니다

9,155줄

import 대응표
옛 경로 → 1.x 경로

42개

메서드 대응표
.run() → .invoke() 등

295개

공식 문서
최신 271 · 가이드 24

11,945개

검색용 조각 (청크)
Chroma Cloud + BM25

출처 4가지

PyPI 패키지 코드	langchain-cli 변환 규칙	최신 공식 문서	0.3.30 옛 마이그레이션 가이드
버전별 도구 목록 42개	공식 자동 변환 규칙	임베딩: text-embedding-3-small	옛 방식 → 새 방식 설명

네 단계를 거치고, 실패하면 되돌아갑니다

질문 분류 → 코드 변환이면 아래 파이프라인 · LangChain 질문은 문서 검색으로 답 · 일반 질문은 바로 답



연결 · 재시도 · 질문 분류: 1번 (LangGraph) · 목표 버전: 질문에서 읽음 (기본 최신 1.4.5) · 모델: gpt-4o-mini

verifier는 LLM 없이 11가지를 검사합니다

1 문법

파이썬 문법 통과

2 근거 ID

보여 준 근거만 썼는지

3 변경 기록

기록 없는 변경 = 환각

4 남은 옛 import

옛 경로가 남았는지

5 import 존재

1.x에 그 경로가 있는지

6 남은 옛 메서드

.run() 같은 옛 호출 (경고)

7 안 쓰는 import

필요 없는 import가 남았나

8 반환값 보존

문자열 결과를 잃지 않았나

9 정의 안 된 이름

실행하면 NameError

10 출력 꺼내기

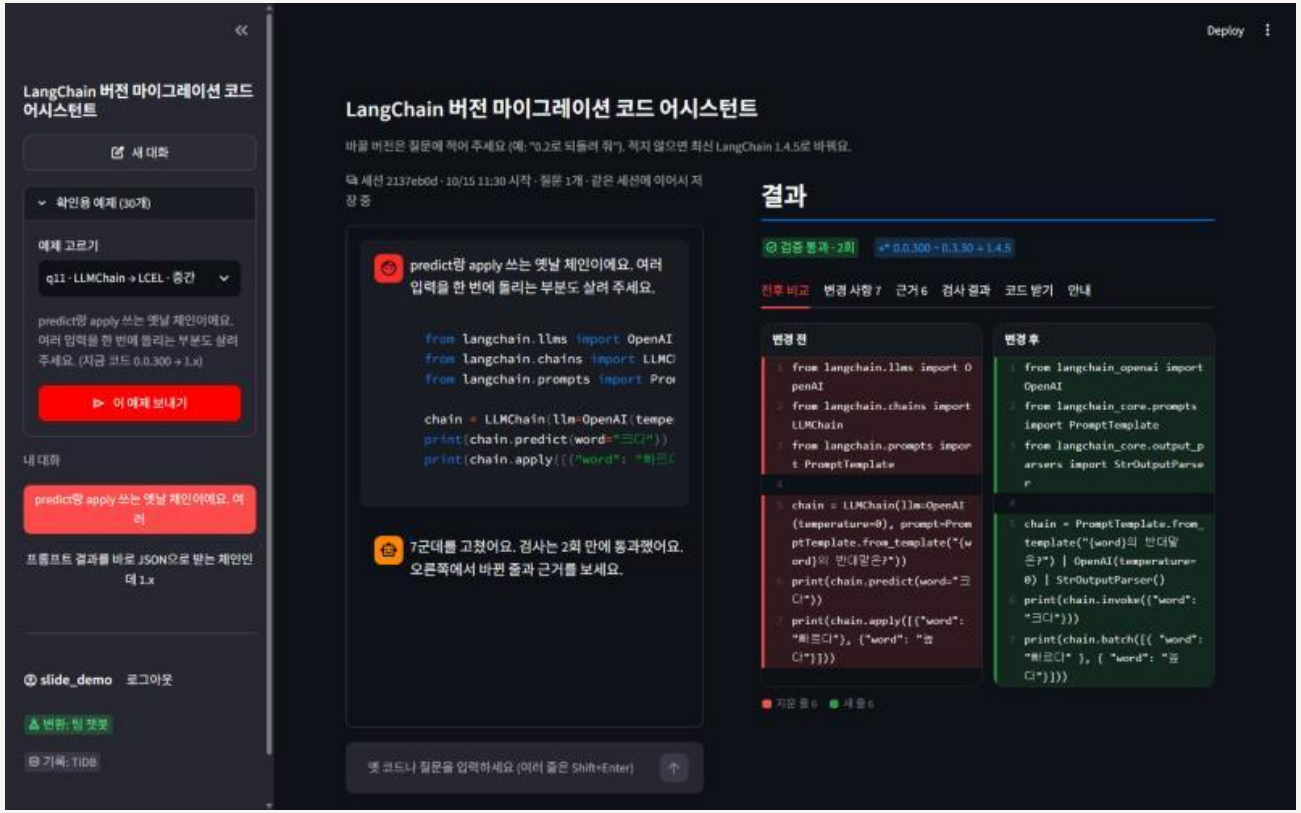
['text'] 잘못 꺼내기

11 체인 입출력 키

실제 키와 같은지

오류 → 다시 시도
경고 → 결과에 표시

결과와 근거를 한 화면에서 봅니다



말로 부탁

"0.2로 되돌려 줘" · 버전 선택 상자 없음

진행 단계 실시간

찾기 → 근거 → 고치기 → 검사 (다시 시도까지)

결과 패널

전후 비교 · 변경 사항 · 근거 · 검사 결과

코드 받기 · 다시 시도

.py · 변경 리포트 .md · 실패 시 버튼 하나로

대화 저장 · 예제 30개

새 대화마다 세션, 사이드바 예제 바로 보내기

TiDB에 사용자와 대화를 저장합니다

users

uuid	VARCHAR(36)	PK
user_id	VARCHAR(100)	UNIQUE
created_at	DATETIME	

1 : N

chat_history

uuid	VARCHAR(36)	PK
user_id	VARCHAR(100)	FK
session_id	VARCHAR(100)	IDX
question	TEXT	
answer	TEXT	
created_at	DATETIME	IDX

아이디 중복 검사

가입 전 확인 + UNIQUE

질문 + 답 = 한 행

새 대화 = 새 session_id

목록 · 다시 열기

session_id로 묶고 시간순으로

키가 없을 때

로컬 저장으로 넘어가고 이유 표시

같은 질문 30개, 같은 조건으로 비교합니다

1.x로 올리기 30개

10가지 유형 × 3: import 경로 · 로더 · 임베딩 · LLMChain → LCEL · 체인 조합 · RAG · 메모리 · 에이전트 · 출력 파서 · 실무 상황

난이도

쉬움 7 · 중간 15 · 어려움 8
실제 사용자처럼 '요청 문장 + 옛 코드'로 구성

미리 확인 (36/36)

옛 코드가 1.x에서 깨지는지, 기대 import가 되는지
+ 버전 오가기 6개 (되돌리기 · 중간 단계)

항목	규칙
비교 대상	일반 LLM vs 우리 챗봇
모델	양쪽 같은 gpt-4o-mini, temperature 0
입력	같은 요청 문장 + 같은 코드
채점	LLM 없이 코드로, 공통 기준 8가지
최대 시도	3회 (우리 챗봇)
지표	공통 기준 통과율 · 재시도 효과 · 자체 평가

LLMChain 코드가 3회차에 통과했습니다

변경 전 (옛 코드)

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

llm = ChatOpenAI(temperature=0)
chain = LLMChain(llm=llm, prompt=PromptTemplate
    .from_template('{word}의 반대말은?'))
print(chain.run(word='크다'))
```

변경 후 (팀 그래프 결과)

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(temperature=0)
chain = PromptTemplate.from_template(
    '{word}의 반대말은?') | llm | StrOutputParser()
print(chain.invoke({'word': '크다'}))
```

3회

검사 통과까지 시도

6줄

바뀐 줄 모두 근거 ID
(imap · mmap · doc)

0줄

근거 부족으로 못 바꾼 줄

약 1분

한 번 변환에 걸린 시간

실제 실행 · 2026-10-15

질문 하나로, 알아서 세 갈래로 답합니다

일반 질문 "안녕하세요"	**2.5초**	일반 답변
LangChain 질문 "0.3 → 1.0, LLMChain은 어떻게 바뀌었어?"	**12.5초**	문서를 찾아 설명
코드 변환 · 최신 옛 코드 + "최신으로 바꿔 줘"	**16초**	목표 1.4.5 · 1회 만에 통과
코드 변환 · 되돌리기 새 코드 + "0.2에서 돌아가게"	**28초**	목표 0.2.17 · 3회 모두 실패

화면 진행 표시 (실제 기록)

✓ 질문 종류: 코드 변환
✓ ① 바꿀 부분 3곳 · 목표 1.4.5
✓ ② 근거 9개
✓ ③ 5곳 고침
✓ ④ 1회차 실패 → 다시
✓ ② 근거 14개
✓ ③ 6곳 고침
✓ ④ 검사 통과 (2회차)

같은 모델인데, 57% 대 0%입니다

우리 챗봇 · 공통 기준 통과

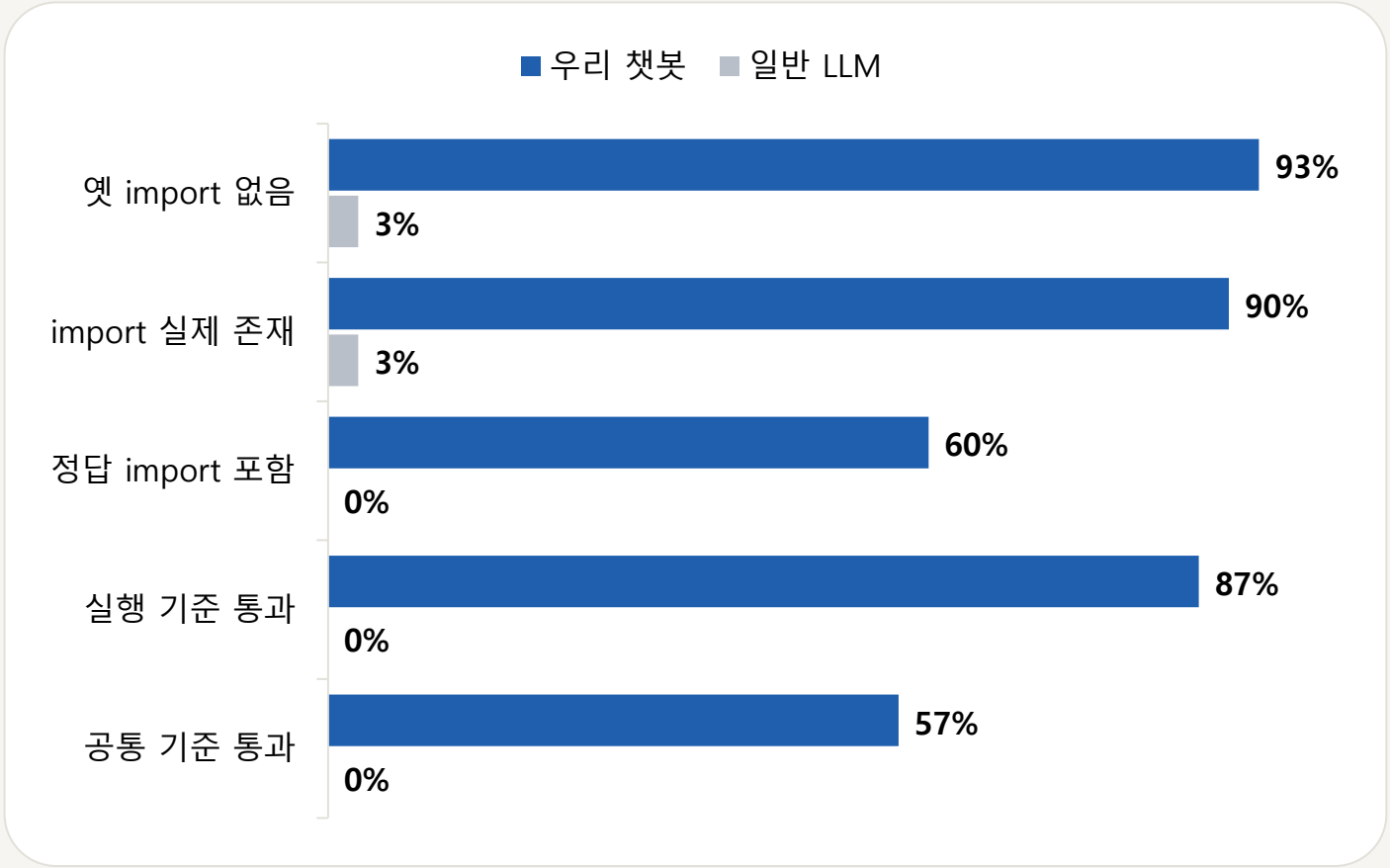
57%

정답 import 빼고 '실행 기준'이면 87%

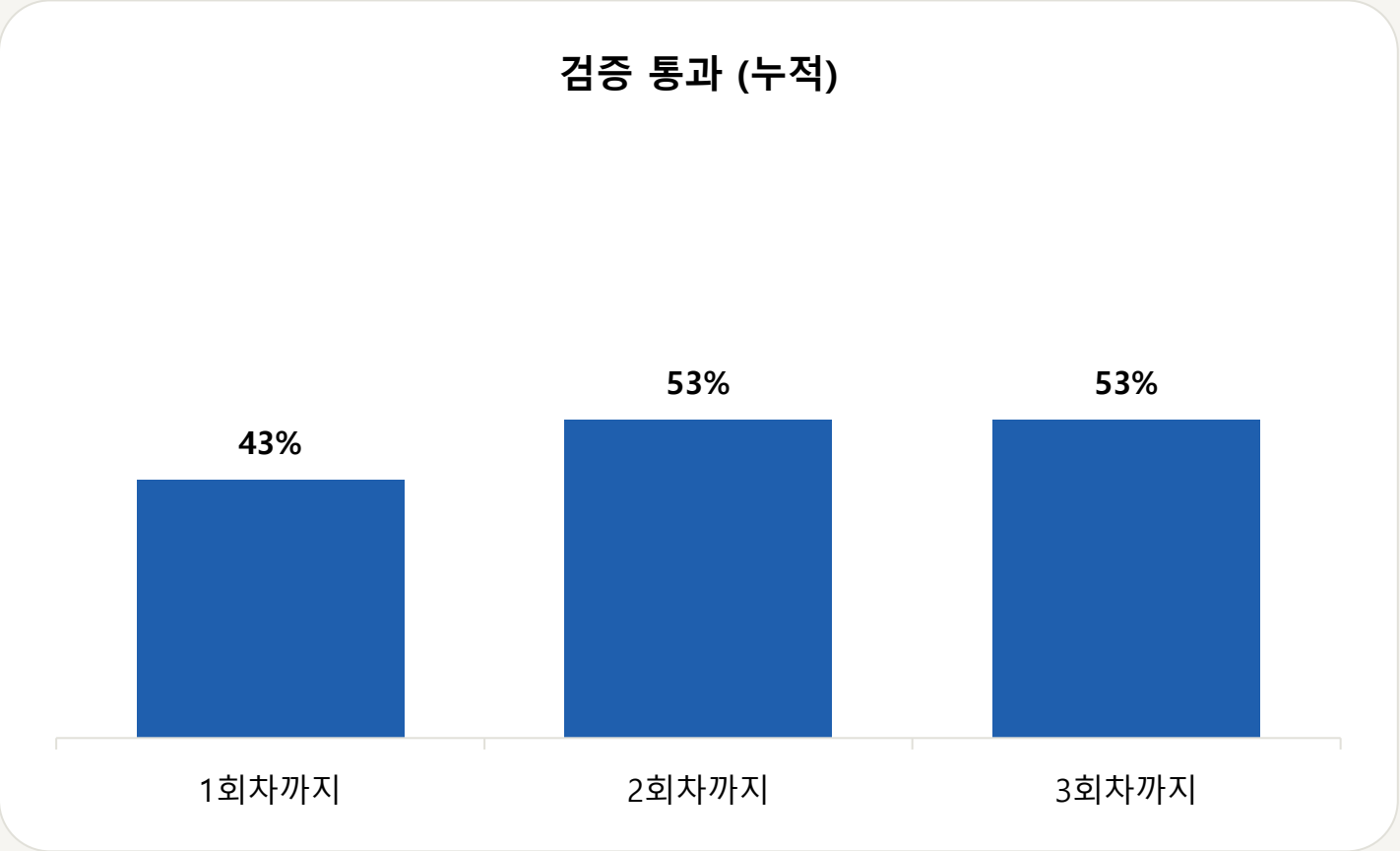
일반 LLM · 공통 기준 통과

0%

30개 중 29개가 옛 import 그대로인데
"1.x에 맞게 수정했다"고 답함



다시 고쳐서 +10%p, 실패는 숨기지 않습니다



1회차 → 최종 검증 통과

43 → 53%

통과한 답의 평균 시도

1.2회

바뀐 줄에 근거가 붙은 비율

93%

실험이 알려 준 약점

약점도 숫자로 찾았고, 고칠 곳도 정했습니다

질문 분류

고쳐 달라면서 에러 이유도 물으면 설명만 하고 끝남 (30개 중 6개).
→ 분류 규칙 보강 · 온도 0

1번 · 분류 확인 3회 모두 같음

환각 · 근거 누락

실패 원인 1위 '기록 없는 변경'. 그래도 검사기가 막아 통과는 못 함.
→ 프롬프트 강화 · 틀린 근거 제외

3번 · 1번

검사기 빈틈

['text'] 꺼내기, 0.x 되돌리기를 1.x 기준으로 검사해 놓침.
→ 목표 버전별 도구 목록으로 검사

3번 · 2번 (데이터는 이미 있음)

옛 창고 우회

최신 방식 대신 langchain_classic으로 우회 3개. 실행은 되지만 지원 종료 예정.
→ 대안 문서 함께 검색 · 안내

2번 · 3번

자체 평가 83점, 약한 곳부터 고칩니다

영역 (팀이 정한 규칙 21가지)	점수	다음 단계 (우선순위 순)
분석 · 근거 검색 · 답변 규칙 · 안정성	98~100%	유지 (목표 달성)
질문 분류	14.0 / 15	① 고쳐 달라는 말이 있으면 코드 변환으로 분류
고쳐 쓰기 (환각 · 근거 누락)	22.3 / 25	② 없던 줄 금지 명시 · 틀린 근거는 다음 시도에서 빼기
검증 · 재시도	17.3 / 30	③ 목표 버전별 검사 · ['text'] 검사 · 실행 테스트

한계: 질문마다 1회 실행 · 코드는 실행하지 않고 정적으로 확인 · 질문 30개는 팀이 직접 구축 (실사용 코드로 확장 예정)

라이브러리가 바뀔 때마다, 근거와 함께 확인된 코드

시간 절감

에러를 하나씩 검색하는 대신, 근거가 붙은 답을 바로 확인

신뢰

같은 모델로 0% → 57%. 바뀐 줄의 93%에 근거, 실패하면 실패라고 알림

확장

대응표 · 문서 · 검사 기준만 바꾸면 다른 라이브러리에도 적용

팀

1번 · [이름]

그래프 · 백엔드

최상위 그래프 · 질문 분류 analyzer · 재시도 판단

해결: 노드 간 필드 · 분기 키 통일

2번 · [이름]

데이터 · 검색

대응표 · 청킹 · Chroma 하이브리드 검색 · 목표 버전 판정

해결: 133MB 벡터 DB → Chroma Cloud

3번 · [이름]

변환 · 검증

State · rewriter verifier 11가지 검사

해결: 옛 근거 ID 복사 · import 오탐 차단

4번 · [이름]

화면 · DB · 평가 · 발표

Streamlit 화면 · TiDB 진행 표시 · 평가 도구

해결: 배포 Secrets · 공정한 채점

감사합니다

질문 받겠습니다

팀 저장소 github.com/SKNETWORKS-FAMILY-AICAMP/SKN35-3rd-4Team
배포 langchain-migration-assistant-btn5chcr8ragnwts68rmhf.streamlit.app

부록 1 · 질문 대응용

LLM + RAG 파이프라인 한 장 정리

오프라인: 근거 만들기 (2번, 1회)

PyPI 코드
도구 목록 42개
+ CLI 규칙 2,650

→

import 대응표 9,155
메서드 대응표 42

→

문서 271 + 옛 가이드 24
MDX 정리 · 제목 분할

→

청크 11,945
최대 4,000자

→

임베딩 1536
→ Chroma Cloud

온라인: 질문 하나 (1 · 2 · 3번)

classifier
migration · search
· general

LLM · 1번

→

① analyzer
코드 · 목표 버전
바꿀 부분 (JSON)

LLM temp 0 · 1번

→

② retriever
정확 매칭 → 메서드
→ 유사 → 하이브리드

RRF k=60 · 상위 5 · 2번

→

③ rewriter
근거만 써서 다시 쓰기
줄마다 근거 ID

LLM temp 0 · 구조화 · 3번

→

④ verifier
LLM 없이 11가지
실패 → ② (최대 3회)

검사만 · 3번

State 하나로 전달: old_code · analysis · target_version → evidences · shown_evidence_ids → new_code · changes(근거 ID) · unresolved
→ verify_result · attempt · exclude_ids(틀린 근거 누적) → 화면 결과 패널 · TiDB

부록 2 · 질문 대응용

파일 구조와 담당

경로	하는 일	담당
src/graph/supervisor.py · migration.py	최상위 그래프(분류 → 세 갈래) · 변환 파이프라인(재시도)	1번
src/graph/nodes/ classifier · general · analyzer	질문 분류 · 일반 답 · 바꿀 부분 찾기	1번
src/rag/ docs_collect · import_map · chunking · embedding · retrieval	문서 수집 · 대응표 · 목표 버전 판정 · 청킹 · Chroma · 하이브리드 검색	2번
src/graph/nodes/ retriever · search	근거 검색 · LangChain 질문 답변	2번
src/graph/state.py · nodes/ rewriter · verifier	State · 근거를 붙여 다시 쓰기 · 11가지 검사	3번
streamlit_app.py · src/ui/	화면 · 그래프 호출 · 진행 표시 · 결과 패널 · TiDB	4번
eval/ · tools/bot_check.py · docs/	평가 질문 · 실행 · 채점 · 화면 점검 · 보고서	4번
data/raw · data/processed	도구 목록 42 · 문서 · 대응표 9,155 · 청크 11,945	2번

부록 3 · 질문 대응용

역할별로 막혔던 것과 푼 방법

1번 · 그래프

analyzer 출력 필드 불일치 → 다음 단계 필드에 맞춤 (PR #7)

분기 키 불일치 → general · search · migration 통일 (PR #14)

LLM이 코드펜스로 감쌈 → 후처리

목표 버전 해석 불가 → 최신 1.4.5로

2번 · 데이터 · 검색

133MB 로컬 DB → Chroma Cloud (PR #10)

re-export · pydantic_v1 경로 → 정의부 추적 · 모듈 매핑

같은 메서드 이름 여러 클래스 → 질의 단서로 순위

다운그레이드 파트너 패키지 → 판정 수정 (PR #13)

3번 · 변환 · 검증

옛 근거 ID 복사 → 보여 준 근거만 인정

venv에서 정상 경로도 import 실패 → 정적 심볼 검사

검증 통과 후 실행 실패 → 9 · 10 · 11번 검사 추가

구조화 출력 누락 → 재시도 · rewrite_failed

4번 · 화면 · 평가

app.py 충돌 → streamlit_app.py 분리

배포 키 오류 → Secrets 먼저 읽기 · 성공만 캐시

1분 대기 → 진행 단계 실시간 표시

공정한 비교 → LLM 없는 공통 채점 8항목